

YubiKit .NET v2

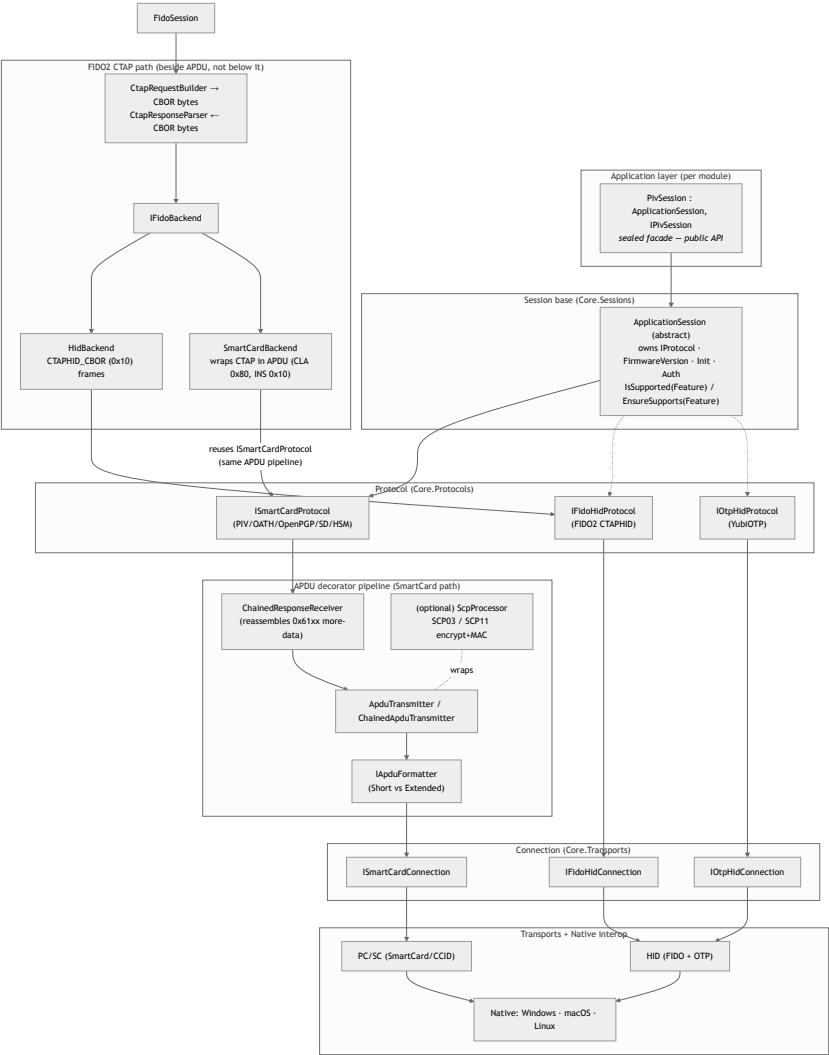
A tour for people who live in the other SDKs

yubikey-manager · yubikit-android · yubikit-swift · python-fido2

- **Async on the golden path** — every applet operation is `async` ; the few sync members that remain are deliberate lifecycle and transaction escapes
- **Install only what you use** — ten packages; all ten assemblies total 1.41 MiB
- **Native AOT** — 11.7 MiB resident, whole SDK linked
- **Event-driven discovery** — OS notifications, not a 500 ms polling timer
- **One session shape** — learn one applet, you know the rest

Branch `yubikit` @ `d04d59aa` · 2026-09-07

Host App → SDK → Session → YubiKey



The pipeline in code

```
// 1. HOST APP asks the SDK for devices
var keys = await YubiKeyManager.FindAllAsync();

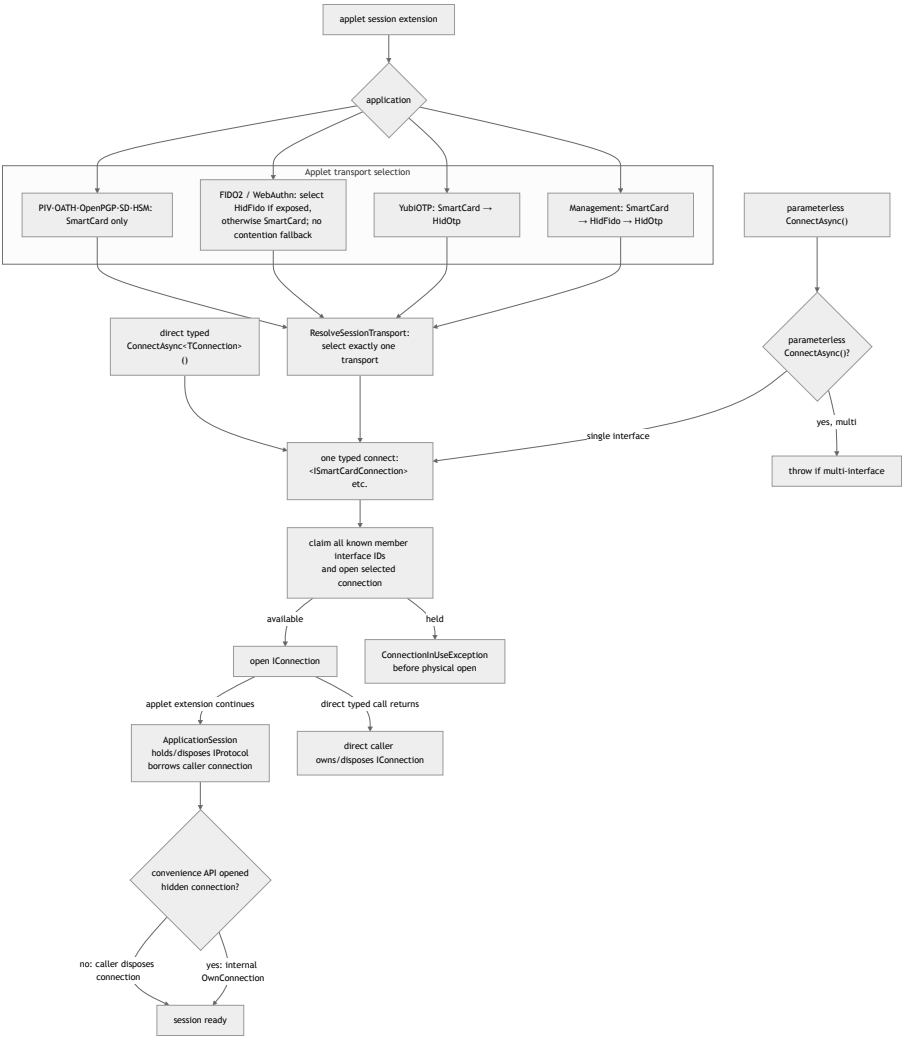
// 2. SDK returns IYubiKey handles
IYubiKey key = keys[0];

// 3. SESSION opened over a transport the SDK resolves
await using var piv = await key.CreatePivSessionAsync();

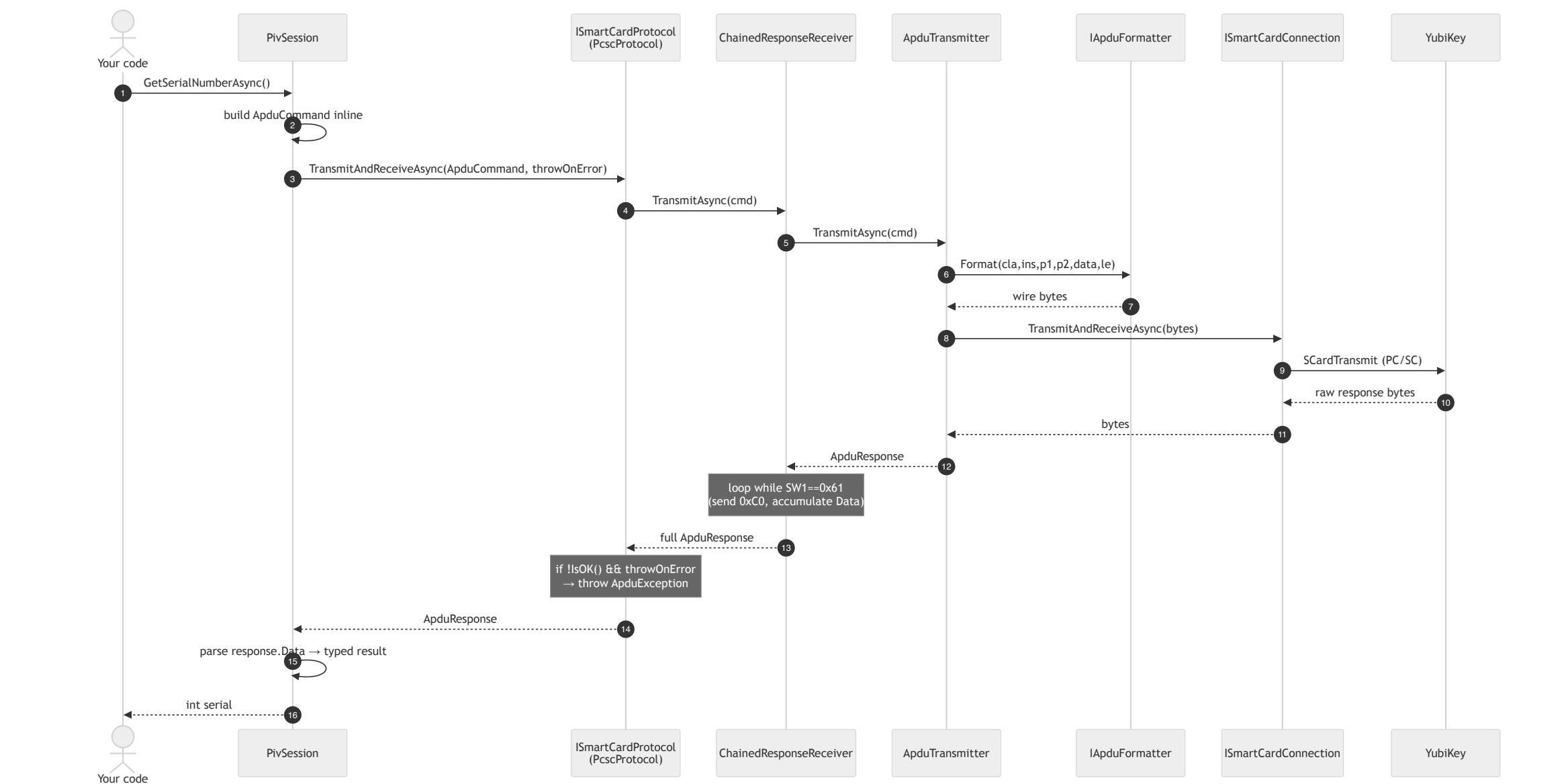
// 4. YUBIKEY executes; the session speaks the applet protocol
var cert = await piv.GetCertificateAsync(PivSlot.Authentication);
```

Every layer boundary is `async`. There is no synchronous escape hatch on the golden path.

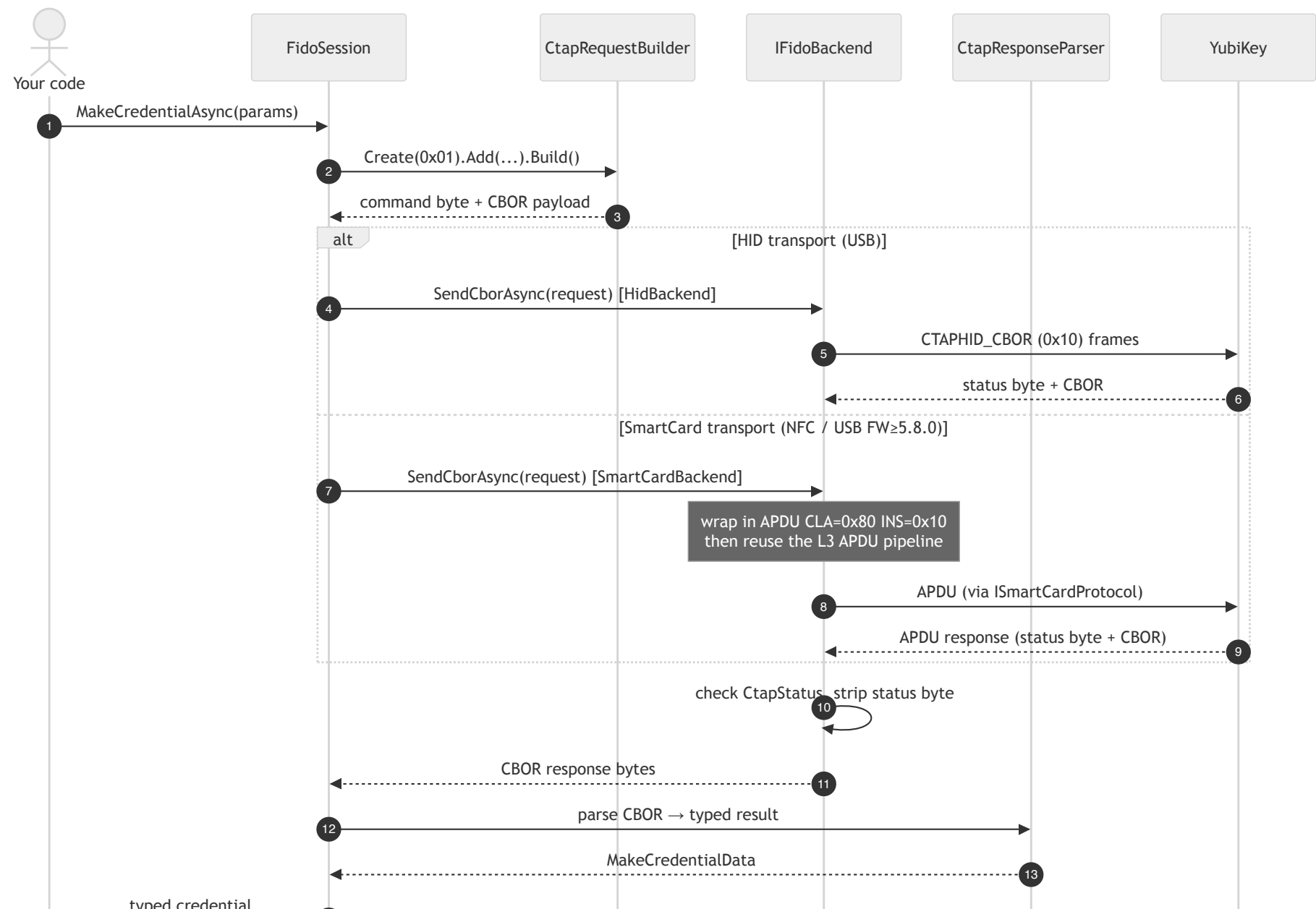
Transports and interfaces



Command invocation: smart card / APDU



Command invocation: FIDO HID / CTAP



Applets are not one-transport-each

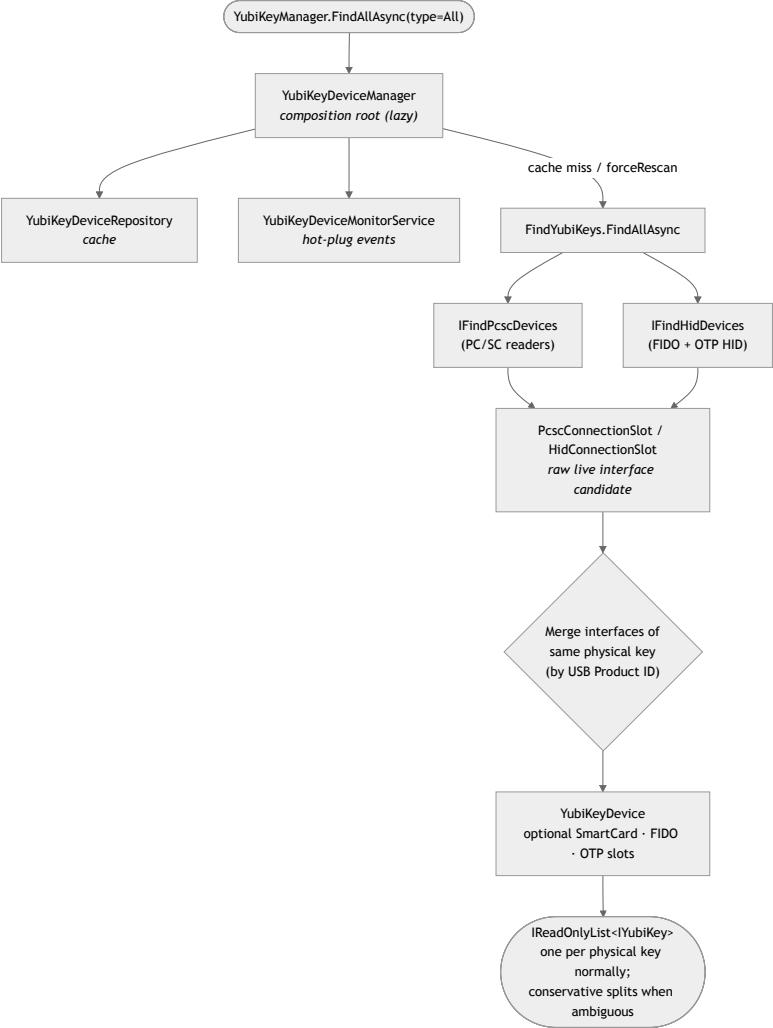
[Flags] ConnectionType : Unknown=0 · Hid=1 · HidFido=2 · Hid0tp=4 · SmartCard=8 · All

Applet	Default transport order
Management	SmartCard → HidFido → Hid0tp — three
FIDO2	HidFido → SmartCard
WebAuthn	inherits FIDO2's order
YubiOTP	SmartCard → Hid0tp
PIV · OATH · OpenPGP · SecurityDomain · YubiHSM	SmartCard only

- FIDO2's SmartCard path is **NFC, or USB-CCID on firmware 5.8.0+**.
- **YubiOTP prefers SmartCard**, not HID — on a CCID-enabled key the YubiOTP snippet later in this deck runs over APDU.
- Supplying `ScpKeyParameters` **forces SmartCard** when no preference is given.

```
await using var mgmt = await key.CreateManagementSessionAsync(  
    new SessionCreationOptions { PreferredConnectionType = ConnectionType.HidFido });
```

Device discovery



Discovery API

```
// Cached – the repository's current view
var keys = await YubiKeyManager.FindAllAsync();

// Force a fresh scan, or narrow by transport
var scan = await YubiKeyManager.FindAllAsync(
    ConnectionType.SmartCard, forceRescan: true);
```

Discovery is **publish-first and degraded-state tolerant** — a key is published as soon as it is enumerated, even if its metadata read has not succeeded.

Canonical Rust instead withholds publication until metadata is read. That difference is deliberate, and it is why `SerialNumber` can arrive late (next slide).

Merging: one physical key, many interfaces

A single YubiKey appears as **several OS-level devices** — a PC/SC reader, a FIDO HID device, an OTP HID device. The SDK merges them into one `IYubiKey`.



Merge inputs per interface: `Connection` · `IsUsb` · `Pid` · `Serial` · `DeviceInfo` · `TopologyKey` (Windows Container ID; `null` on macOS and Linux)

NFC never merges. Neither does a PC/SC reader of unknown kind — only USB-attached interfaces are merge candidates. Correlation uses an internal, machine-local interface-set key.

Identity: what is stable, what is not

Surface	Guarantee
DeviceId	Diagnostic only. Not durable identity.
SerialNumber	Latched. null → value, never back to null.
Interface-set key	Internal, machine-local, never public
Equals / GetHashCode	Referential — same object, or not equal

- SerialNumber may stay null **forever**: reads fail, budgets exhaust, and the Security Key series reports no serial at all.
- It can flip null → value **after** publication, with **no** device event.
- A key whose interface set changes is **republished as a new object** that inherits nothing from its predecessor.

Full DeviceInfo stays internal: capabilities, flags and config are mutable via Management, so a cached copy goes stale. The serial is the one burned-in field.
Need the rest? await key.GetDeviceInfoAsync() reads it live.

Observability: the whole device-event API

```
YubiKeyManager.StartMonitoring(); // events don't flow until this

await foreach (var e in YubiKeyManager.WatchAsync())
{
    Console.WriteLine($"{e.Action}: {e.Device.DeviceId}");
}
```

```
public enum DeviceAction { Added, Removed }
public class DeviceEvent { IYubiKey Device; DeviceAction Action; DateTime Timestamp; }
```

The monitoring **control** surface is five members: `StartMonitoring()`, `StartMonitoring(TimeSpan)`, `StopMonitoring()`, `WatchAsync(ct)`, `IsMonitoring`. `Shutdown()` / `ShutdownAsync()` also stop monitoring as part of tearing the manager down. **No `IObservable`**. No Rx dependency. BCL types only.

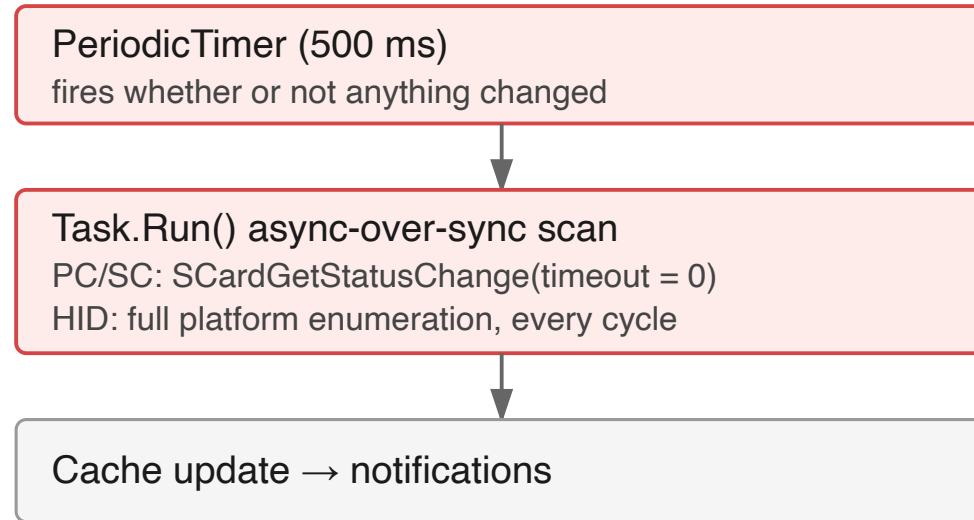
How the peers surface device changes

SDK	Attach / detach
.NET v2	<code>await foreach WatchAsync()</code> — OS-notification driven
<code>yubikey-manager</code> (Python)	Poll only — <code>scan_devices()</code> in a <code>sleep</code> loop
<code>yubikey-manager</code> (rust)	<code>monitor_yubikeys(cb)</code> — <code>udev</code> / <code>WM_DEVICECHANGE</code> / <code>IOHIDManager</code>
<code>yubikit-android</code>	Callbacks — <code>Callback<T>.invoke</code> <code>attach</code> , <code>setOnClosed</code> <code>detach</code>
<code>yubikit-swift</code>	Connect-on-demand — <code>makeConnection()</code> polls; <code>waitUntilClosed()</code>
<code>python-fido2</code>	None — <code>list_devices()</code> enumerates on call

Two things worth noting: rust has a third variant, `YubiKeyEvent::Changed`, that .NET does not. And Swift's `makeConnection()` is a 1-second poll loop internally, not an OS notification.

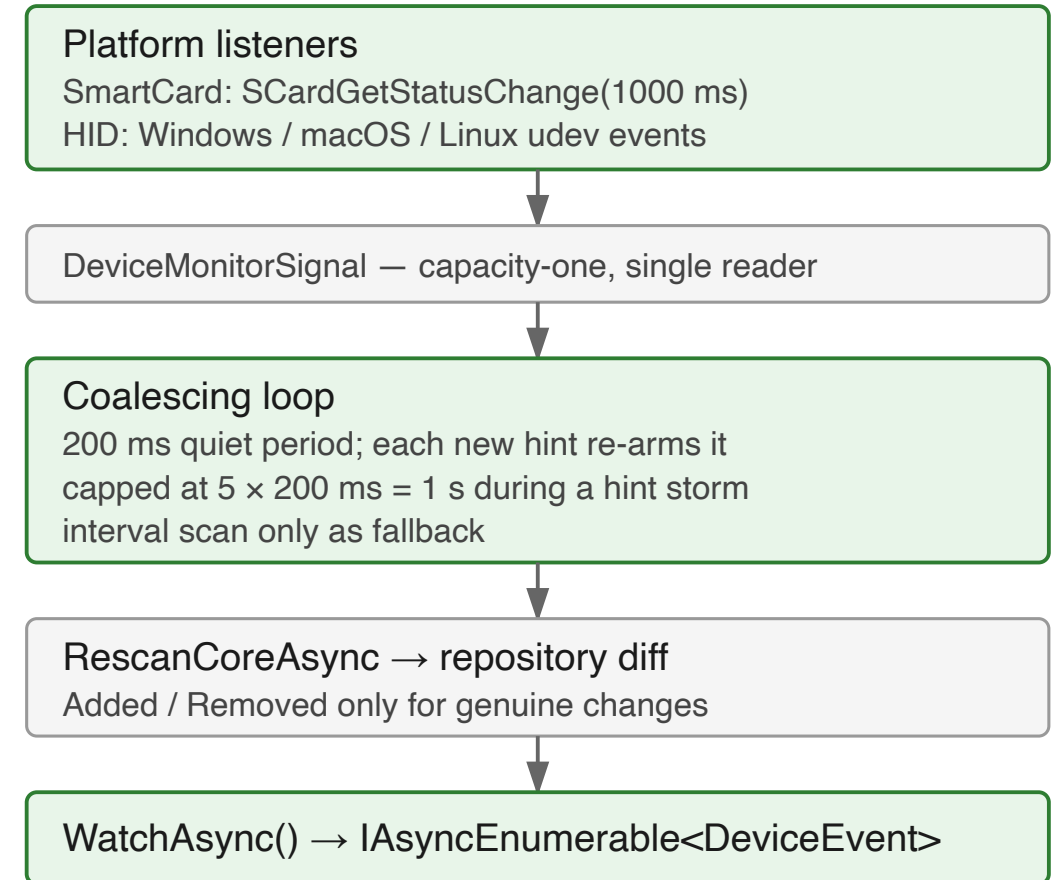
Observability: what changed underneath

v1 — timer polling



- ✗ 500 ms detection latency
- ✗ CPU burned polling an idle bus
- ✗ full re-enumeration every cycle
- ✗ Windows HID not implemented
- ✗ async-over-sync anti-pattern

v2 — OS notifications + coalescing



Observability: three contracts worth knowing

1 — `WatchAsync` subscribes on first iteration, not when called.

Start the `await foreach` *before* the action you expect to trigger an event, or you will miss anything raised in the gap.

2 — Overflow faults the stream; it does not drop.

Each enumeration owns a 256-event buffer. Overflow throws `InvalidOperationException` on *that* stream only. Device events are **deltas**, so a silent drop would desynchronise you — recover by re-enumerating and calling `FindAllAsync`.

3 — `StartMonitoring(interval)` while already running is a silent no-op.

The new interval is ignored, not applied, and no error is raised. Stop and start to change it. Deliberate: partial application would be worse, and throwing would make an idempotent start unsafe to call defensively.

Logging is opt-in and static: `YubiKitLogging.Configure(loggerFactory)`, one line, before you touch the SDK. **Never inject `ILogger`** — that house rule is what keeps the SDK usable without a DI container.

Going below the applet API: three tiers

Tier	What it gives you	What you take on
0 — Applet session	applet semantics, feature gates, protocol state	nothing
1 — Raw session	APDU/CTAP framing, chaining, overlap refusal, SCP	payload semantics
2 — Raw connection	a pipe	<i>everything</i>

```
await using var piv  = await key.CreatePivSessionAsync();           // Tier 0
await using var raw  = await key.CreateRawSmartCardSessionAsync(); // Tier 1
await using var conn = await key.ConnectAsync<ISmartCardConnection>();
var rsp = await conn.TransmitAndReceiveAsync(rawBytes);             // Tier 2
```

Tier 1 does **not** select an applet or apply feature gates. At Tier 2 you own APDU formatting, chaining, response correlation, CRC validation, keep-alive, sequencing, and recovery from partial or cancelled exchanges.

Dropping a tier does **not** drop transport safety — raw sessions still obey one-connection-per-key and one-session-per-connection.

This is the supported answer for teammates hand-rolling APDUs in `ykman` today: there is a place to do that, and it is Tier 1, not Tier 2.

One session shape — for eight of the nine

```
await using var s = await key.CreateXSessionAsync(options, cancellationToken);
```

- A sealed `XSession` plus a matching `IXSession` interface
- `IYubiKey.CreateXSessionAsync(...)` — session owns a hidden connection
- `XSession.CreateAsync(connection, ...)` — session borrows yours
- Both take `SessionCreationOptions?` then a defaulted `CancellationToken`
- Always `await using`

Test-enforced, not conventional — `AppletSessionShapeTests` enumerates the eight sessions and asserts the shape holds for each.

WebAuthn is the deliberate exception. It is not an applet session. It returns a `WebAuthnClient`, takes a required origin and public-suffix checker, and has its own factory test. The test file says so in as many words:

"WebAuthn is not an applet session and has its own factory test."

Ownership: who disposes the connection

```
// Convenience – the session owns a hidden connection and disposes it
await using var piv = await key.CreatePivSessionAsync();
```

```
// Direct factory – you own the connection, you dispose both
await using var conn = await key.ConnectAsync<ISmartCardConnection>();
await using var mgmt = await ManagementSession.CreateAsync(conn);
var info = await mgmt.GetDeviceInfoAsync();
```

One key admits one live connection. One connection admits one live session — a second `Attach` throws `ConnectionInUseException`. Dispose session N before creating N+1 over the same connection. Prefer `DisposeAsync`; sync `Dispose` blocks to drain.

SDK	Who owns the connection
.NET v2	Both, named explicitly at the call site
ykman (Python)	Caller — <code>with dev.open_connection(...)</code>
ykman (rust)	Session consumes it; returns <code>(Error, Connection)</code> on failure
yubikit-android	Session borrows, but <code>close()</code> closes the connection too
yubikit-swift	Caller owns; session never closes it

Management

Read device info, toggle capabilities, write device config.

```
var info = await key.GetDeviceInfoAsync(); // no session needed
```

ykman (Python)

```
mgmt = ManagementSession(conn)
info = mgmt.read_device_info()
```

yubikit-swift

```
let s = try await Management.Session.makeSession(connection: connection)
let info = try await s.getDeviceInfo()
```

Delta: .NET gives Management a device-level shortcut — no session boilerplate for the most common read. It is also the widest applet: three transports, `SmartCard` → `HidFido` → `Hid0tp`. Swift covers two, with `SmartCardConnection` and `FIDOConnection` overloads.

PIV

Certificates, key pairs, PIN/PUK, attestation.

```
await using var piv = await key.CreatePivSessionAsync();  
var cert = await piv.GetCertificateAsync(PivSlot.Authentication);
```

ykman (Python)

```
piv = PivSession(conn)  
cert = piv.get_certificate(SLOT.AUTHENTICATION)
```

yubikit-android

```
PivSession piv = new PivSession(smartCardConnection);  
X509Certificate cert = piv.getCertificate(Slot.AUTHENTICATION);
```

Delta: all three return a real certificate type — `X509Certificate2`, `x509.Certificate`, `java.security.cert.X509Certificate`. The .NET difference is **nullability**: an empty slot returns `null` rather than throwing.

OATH

TOTP and HOTP credentials, calculate codes, applet password.

```
await using var oath = await key.CreateOathSessionAsync();  
var codes = await oath.CalculateAllAsync();
```

ykman (Python)

```
oath = OathSession(conn)  
creds = oath.list_credentials()
```

yubikit-android

```
OathSession oath = new OathSession(conn);  
Map<Credential, @Nullable Code> codes = oath.calculateCodes();
```

Delta: the closest convergence in the deck — .NET's

`IReadOnlyDictionary<Credential, Code?>` and Android's `Map<Credential, @Nullable Code>`

express the same touch-required semantics. .NET makes it a compiler-enforced `Code?`

rather than an annotation.

OpenPGP

Card data, key slots, sign / decrypt / authenticate, PIN management.

```
await using var pgp = await key.CreateOpenPgpSessionAsync();  
var data = await pgp.GetApplicationRelatedDataAsync();
```

ykman (Python)

```
pgp = OpenPgpSession(conn)  
data = pgp.get_application_related_data()
```

yubikit-android

```
OpenPgpSession pgp = new OpenPgpSession(conn);  
ApplicationRelatedData d = pgp.getApplicationRelatedData();
```

Delta: `yubikit-swift` has **no OpenPGP session at all** — only a `Capability.openPGP` bit. If you work in Swift, this applet is entirely new to you.

FIDO2 — the CTAP2 layer

Raw CTAP2: `GetInfo`, `MakeCredential`, `GetAssertion`, credential management, bio.

```
await using var fido = await key.CreateFidoSessionAsync();  
var info = await fido.GetInfoAsync();
```

python-fido2

```
ctap2 = Ctap2(dev)  
info = ctap2.info      # cached at construction
```

yubikit-swift

```
let s = try await CTAP2.Session.makeSession(connection: connection)  
let info = try await s.getInfo()
```

Delta: `python-fido2` runs `GET_INFO` **inside the constructor** and caches it; .NET keeps construction cheap and the round-trip explicit and cancellable. Note FIDO2 is dual-transport: `HidFido` first, then SmartCard — NFC, or USB-CCID on **firmware 5.8.0+**.

WebAuthn — the layer above CTAP2

Origin checks, client data, attestation, extensions, PIN/UV orchestration.

```
_ = WebAuthnOrigin.TryParse("https://example.com", out var origin);  
await using var client = await key.CreateWebAuthnClientAsync(  
    origin!, isPublicSuffix: d => d is "com" or "org" or "net");  
var reg = await client.MakeCredentialAsync(options, pinBytes);
```

python-fido2

```
result = client.make_credential(create_options["publicKey"])
```

yubikit-swift (*release/1.4.0*)

```
let client = WebAuthn.Client(session: session, origin: try .init("https://example.com"),  
                             isPublicSuffix: { publicSuffixList.contains($0) })  
let r = try await client.makeCredential(options, authorization: .pin("1234")).value
```

Delta: both .NET and Swift demand an **origin** and a **public-suffix checker up front** — you cannot accidentally skip origin validation. `python-fido2` also ships the relying-party half (`Fido2Server`); v2 is client-side only.

SecurityDomain (SCP)

GlobalPlatform key management — SCP03 and SCP11, certificates, CA identifiers.

```
await using var sd = await key.CreateSecurityDomainSessionAsync();  
var certs = await sd.GetCertificatesAsync(keyReference);
```

ykman (Python)

```
sd = SecurityDomainSession(conn)  
keys = sd.get_key_information()
```

yubikit-swift

```
let s = try await SecurityDomainSession.makeSession(connection: connection)  
let keyInfo = try await s.getKeyInformation()
```

Delta: in .NET, SCP is *also* a creation option on every other applet — pass

`SessionCreationOptions { ScpKeyParameters = ... }` and any session runs over a secure channel. Supplying it without a transport preference **forces SmartCard**.

YubiHSM Auth

Store and use credentials that authenticate to a YubiHSM 2.

```
await using var hsm = await key.CreateHsmAuthSessionAsync();  
var creds = await hsm.ListCredentialsAsync();
```

ykman (Python)

```
hsm = HsmAuthSession(conn)  
creds = hsm.list_credentials()
```

ykman (rust)

```
let mut s = HsmAuthSession::new(conn).map_err(|(e, _)| e)?;  
let creds = s.list_credentials()?;
```

Delta: only .NET, Python and rust implement this. `yubikit-android` has **no** YubiHSM Auth module; `yubikit-swift` has only the `Capability.hsmAuth` bit.

YubiOTP

The two programmable slots — Yubico OTP, static password, HMAC-SHA1 challenge-response.

```
await using var otp = await key.CreateYubi0tpSessionAsync();  
var serial = await otp.GetSerialNumberAsync();
```

ykman (Python)

```
otp = Yubi0tpSession(conn)  
state = otp.get_config_state()
```

yubikit-android

```
Yubi0tpSession otp = new Yubi0tpSession(otpConnection);  
byte[] r = otp.calculateHmacSha1(Slot.TW0, challenge, null);
```

Delta: despite the name, .NET's YubiOTP session is **dual-transport and prefers**

SmartCard — `SmartCard` → `Hid0tp` . On a CCID-enabled key the snippet above runs over APDU, not HID. `yubikit-swift` has no YubiOTP slot session at all.

Applet coverage across the SDKs

Applet	.NET v2	ykman (py)	ykman (rust)	android	swift
Management	✓	✓	✓	✓	✓
PIV	✓	✓	✓	✓	✓
OATH	✓	✓	✓	✓	✓
OpenPGP	✓	✓	✓	✓	✗
FIDO2 / CTAP2	✓	➡	✓	✓	✓
WebAuthn	✓	➡	✓	✓	✓
SecurityDomain	✓	✓	✓	✓	✓
YubiHSM Auth	✓	✓	✓	✗	✗
YubiOTP	✓	✓	✓	✓	✗



= not implemented in-repo; delegated to `python-fido2`, a hard dependency (`fido2 >=2.0, <3`). That library supplies both `Ctap2` and `Fido2Client` — and, uniquely, `Fido2Server`, the relying-party half no other SDK here ships.

.NET v2 and the rust experiment are the only two with all nine in one repo.

Footprint: what you ship

Managed assemblies (Release, net10.0) — install only what you use:

Assembly	KiB		Assembly	KiB
Core	577		WebAuthn	130
Fido2	197		OpenPgp	110
Piv	165		Oath	62
SecurityDomain	56		YubiOtp	56
YubiHsm	51		Management	37

All ten: 1,441 KiB = 1.41 MiB. A PIV-only app pays Core + Piv = 742 KiB.

These are assembly sizes from bin/Release/net10.0 , not .nupkg sizes.

Native AOT, verification host linking all ten libraries:

	Native AOT	Framework-dependent
Executable	3.11 MiB	—
NativeShims sidecar	3.71 MiB	3.71 MiB
Peak RSS	11.7 MiB	51.5 MiB → 4.4× less
CPU (user + sys)	20 ms	170 ms

Footprint: how these were collected

Machine. Apple M1, 8 cores, 16 GB, macOS 15.7.7, .NET SDK 10.0.100, RID `osx-arm64` .
Repo @ `d04d59aa` . **6 YubiKeys physically attached.**

Number	Provenance
Assembly sizes	Measured — <code>stat</code> on <code>bin/Release/net10.0/*.dll</code>
AOT exe, RSS, CPU, wall	Measured — <code>dotnet publish -r osx-arm64 -p:PublishAot=true</code> of <code>verification/NativeAotVerification</code> ; <code>/usr/bin/time -l</code> ; wall = median of 10 after 3 warm-ups; RSS and CPU from a single representative run
Framework-dependent baseline	Measured — same project, <code>-p:PublishAot=false --self-contained false</code>
AOT link coverage, support contract	Quoted — PR #578, <code>docs/NATIVE-AOT.md</code>
Recurring AOT CI	CI — <code>native-aot.yml</code> run <code>34121554932</code> on <code>yubikit</code> , macOS arm64, hardware-free (asserts <code>Found 0 YubiKey(s)</code>)

 **Two caveats.** Wall time is dominated by I/O enumerating six attached keys, not by startup: subtracting CPU leaves **508 ms** (AOT) and **458 ms** (framework-dependent) of non-CPU time — same order, ~11 % apart, and both far larger than either CPU figure. Second, `/usr/bin/time -l` quantises CPU to 10 ms, so "20 ms vs 170 ms" is two ticks against seventeen: **treat the CPU ratio as "roughly an order of magnitude", not 8.5x.**

Takeaways

One shape, eight applet sessions — test-enforced, not conventional.

WebAuthn is the deliberate exception: it returns a client, not a session.

Ownership is explicit. Convenience owns the connection, the direct factory borrows it.

One connection, one live session, enforced at runtime.

Device events are one API. `StartMonitoring()` + `await foreach WatchAsync()`.

No Rx, no `IObservable`, BCL types only.

Identity is honest about what it cannot promise. `DeviceId` is diagnostic;

`SerialNumber` may be `null` forever and can arrive late without an event.

AOT is real. 11.7 MiB resident with the whole SDK linked, ~4.4× less than JIT.

`docs/architecture/` · `docs/usage/device-discovery.md` · `docs/NATIVE-AOT.md`

Status **2.0.0-alpha.2** — public API still in `PublicAPI.Unshipped.txt`, so breaking

changes are still cheap. Now is the time to complain.

`<script src="assets/vendor/panzoom.min.js"></script>` `<script src="assets/deck-zoom.js"></script>`